



# OSR eHSM 安全启动与升级镜像生成工具用户使用指南

|    |                |
|----|----------------|
| 版本 | 1.1            |
| 作者 | OSR IP         |
| 部门 | OSR IP         |
| 日期 | 2026 - 02 - 11 |
| 分类 | 用户手册           |

## Copyright Notice and Proprietary Information Notice

Copyright © 2014 - 2026 Open Security Research, Inc. All rights reserved. This documentation contains confidential and proprietary information that is the property of Open Security Research, Inc. The documentation is transported under a license agreement and may be used or copied only in accordance with the terms of the license agreement. No part of the software and documentation may be reproduced, transmitted, translated, distributed, disclosed, announced or copied in any form or by any means, electronic, mechanical, manual, optical, or otherwise, without prior written permission of Open Security Research, Inc., or as expressly provided by the license agreement.

## 版权声明和专有信息声明

版权所有 © 2014 - 2026 深圳市组创信安科技开发有限公司 保留所有权利。本文档包含机密和专有信息，属于深圳市组创信安科技开发有限公司的财产。该文档根据对应的许可协议传播，且只能根据许可协议的条款使用或复制。未经深圳市组创信安科技开发有限公司事先书面许可或经许可协议明确规定，该文档和对应软件的全部或任何部分均不得以任何形式或方式（电子的，机械的，手动的，光学的或其他的）被复制、传播、翻译、分发、披露、宣布或抄袭。

## 修订历史

| 版本  | 修改日期       | 描述                                                                                      |
|-----|------------|-----------------------------------------------------------------------------------------|
| 1.0 | 2026-01-16 | 初始版本                                                                                    |
| 1.1 | 2026-02-11 | 新增 C 数组导出功能 (CLI <code>-c</code> 选项及 C 库 <code>gen_image_c_array_from_config</code> 接口) |

## 目录

|                                                 |    |
|-------------------------------------------------|----|
| 1 简介 .....                                      | 5  |
| 1.1 功能概述 .....                                  | 5  |
| 1.2 接口形式 .....                                  | 5  |
| 1.3 支持的算法 .....                                 | 5  |
| 2 快速开始 .....                                    | 6  |
| 2.1 准备工作 .....                                  | 6  |
| 2.2 生成 Boot 镜像 .....                            | 6  |
| 2.3 生成 Upgrade 镜像 .....                         | 6  |
| 3 CLI 工具使用 .....                                | 8  |
| 3.1 命令格式 .....                                  | 8  |
| 3.2 参数说明 .....                                  | 8  |
| 3.3 使用示例 .....                                  | 8  |
| 4 C 库接口 .....                                   | 10 |
| 4.1 头文件 .....                                   | 10 |
| 4.2 错误码 .....                                   | 10 |
| 4.3 API 函数 .....                                | 10 |
| 4.3.1 gen_image_from_config .....               | 10 |
| 4.3.2 get_image_size_from_config .....          | 11 |
| 4.3.3 get_imgtool_lib_version .....             | 11 |
| 4.3.4 gen_image_c_array_from_config .....       | 11 |
| 4.4 使用示例 .....                                  | 12 |
| 4.5 编译链接 .....                                  | 13 |
| 5 配置文件格式 .....                                  | 14 |
| 5.1 通用配置 [general] .....                        | 14 |
| 5.2 Boot 模式配置 [boot] .....                      | 14 |
| 5.3 Upgrade 模式配置 [upgrade] .....                | 16 |
| 5.3.1 Boot 层配置 [upgrade.boot_layer] .....       | 16 |
| 5.3.2 Upgrade 层配置 [upgrade.upgrade_layer] ..... | 16 |
| 6 密钥文件格式 .....                                  | 18 |
| 6.1 对称密钥 .....                                  | 18 |
| 6.2 非对称密钥 .....                                 | 18 |
| 6.2.1 RSA 密钥 .....                              | 18 |
| 6.2.2 ECC 密钥 .....                              | 19 |
| 6.2.3 SM2 密钥 .....                              | 19 |
| 7 发布包目录结构 .....                                 | 20 |
| 7.1 目录说明 .....                                  | 20 |
| A 镜像结构 .....                                    | 21 |
| A.1 Boot 镜像结构 .....                             | 21 |
| A.2 Upgrade 镜像结构 .....                          | 21 |
| B 版本计数器 .....                                   | 22 |
| C 初始化向量 (IV) .....                              | 23 |

# 1 简介

## 1.1 功能概述

OSR eHSM 镜像生成工具 (imgtool) 是一款用于生成嵌入式硬件安全模块 (eHSM) 安全启动和升级镜像的工具。该工具支持多种加密算法, 可生成符合 eHSM 安全规范的固件镜像。

主要功能包括:

- **Boot 镜像生成:** 生成安全启动镜像, 支持签名和加密
- **Upgrade 镜像生成:** 生成安全升级镜像, 采用双层签名加密机制
- **多算法支持:** 支持 AES、SM4 对称加密和 RSA、ECC、SM2 非对称签名
- **灵活配置:** 通过 TOML 配置文件管理所有参数

## 1.2 接口形式

本工具提供两种接口形式:

- **命令行工具 (CLI):** 适用于开发调试和脚本集成
- **C 静态库:** 适用于嵌入式系统集成和自动化构建

## 1.3 支持的算法

| 类型    | 算法                 | 用途    |
|-------|--------------------|-------|
| 对称算法  | AES128             | 签名/加密 |
| 对称算法  | AES256             | 签名/加密 |
| 对称算法  | SM4                | 签名/加密 |
| 非对称算法 | RSA2048            | 签名    |
| 非对称算法 | RSA3072            | 签名    |
| 非对称算法 | ECC256 (secp256r1) | 签名    |
| 非对称算法 | SM2                | 签名    |

## 2 快速开始

### 2.1 准备工作

使用本工具前，需要准备以下文件：

1. 固件二进制文件：待打包的固件数据（如 `firmware.bin`）
2. 密钥文件：用于签名和加密的密钥（TOML 格式）
3. 配置文件：描述镜像生成参数（TOML 格式）

### 2.2 生成 Boot 镜像

以下是生成 Boot 镜像的基本步骤：

#### 步骤 1：创建密钥文件

创建 AES128 密钥文件 `aes128.toml`：

```
1| type = "aes128"  
2| value = "32BB3737FB468C4B6AC9D84C1661737D"
```

#### 步骤 2：创建配置文件

创建 Boot 配置文件 `boot_config.toml`：

```
1| [general]  
2| mode = "boot"  
3| output = "boot_image.bin"  
4| input_bin = "firmware.bin"  
5| image_type = 0  
6| version_counter = "FFFFFFFFFFFFFFFFFFFFFFFF"  
7|  
8| [boot]  
9| sign_key = "aes128.toml"  
10| enc_key = "aes128.toml"  
11| iv = "5ED24925293ACB755C50D8B263A2A3CC"
```

#### 步骤 3：生成镜像

```
1| imgtool boot_config.toml
```

生成的镜像文件为 `boot_image.bin`。

### 2.3 生成 Upgrade 镜像

Upgrade 镜像采用双层结构，需要分别配置 Boot 层和 Upgrade 层的密钥。

创建 Upgrade 配置文件 `upgrade_config.toml`:

```
1| [general]
2| mode = "upgrade"
3| output = "upgrade_image.bin"
4| input_bin = "firmware.bin"
5| image_type = 0
6| version_counter = "FFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFF"
7|
8| [upgrade.boot_layer]
9| sign_key = "aes128.toml"
10|
11| [upgrade.upgrade_layer]
12| sign_key = "aes128.toml"
13| enc_key = "aes128.toml"
14| iv = "5ED24925293ACB755C50D8B263A2A3CC"
```

执行生成命令:

```
1| imgtool upgrade_config.toml
```

## 3 CLI 工具使用

### 3.1 命令格式

```
1| imgtool <CONFIG> [OPTIONS]
```

### 3.2 参数说明

| 参数                    | 说明                | 示例              |
|-----------------------|-------------------|-----------------|
| <CONFIG>              | 配置文件路径（必需）        | config.toml     |
| -i, --input <FILE>    | 输入固件文件路径，覆盖配置文件设置 | -i firmware.bin |
| -o, --output <FILE>   | 输出文件路径，覆盖配置文件设置   | -o image.bin    |
| -c, --output-c <FILE> | 导出 C 数组源文件到指定路径   | -c image.c      |
| -f, --force           | 强制覆盖已存在的输出文件      | -f              |
| -v, --verbose         | 详细输出模式            | -v              |
| -h, --help            | 显示帮助信息            | --help          |
| -V, --version         | 显示版本信息            | --version       |

### 3.3 使用示例

基本使用：

```
1| imgtool boot_config.toml
```

指定输出文件：

```
1| imgtool boot_config.toml -o output/my_image.bin
```

强制覆盖并显示详细信息：

```
1| imgtool boot_config.toml -f -v
```

导出 C 数组源文件：

```
1| imgtool boot_config.toml -c boot_image.c
```

生成的 .c 文件包含带字段注释的 C 数组，格式如下：

```
1| /* generated by ehsm_imgtool v0.1.0 at 2026-01-01 12:00:00 */
2|
```



```
3| /* clang-format off */
4| const unsigned char g_boot_image[4096] = {
5|     /* signature*/
6|     /* 000000 */ 0xd4, 0xf5, 0x1f, ...
7|     ...
8|     /* code */
9|     /* 000400 */ 0xa6, 0x38, 0x6f, ...
10|    ...
11| };
12| /* clang-format on */
```

变量名从 .c 文件名自动推导：文件名中的非字母数字字符替换为下划线，并添加 `g_` 前缀。例如 `boot_image.c` 生成 `g_boot_image`。

## 4 C 库接口

C 静态库适用于需要在嵌入式系统或自动化构建流程中集成镜像生成功能的场景。

### 4.1 头文件

使用前需包含头文件：

```
1| #include "imgtool_clib.h"
```

### 4.2 错误码

所有 API 函数返回 `EhsmErrorCode` 枚举值：

| 错误码                          | 值 | 说明        |
|------------------------------|---|-----------|
| <code>Success</code>         | 0 | 操作成功      |
| <code>NullPointer</code>     | 1 | 传入了空指针    |
| <code>InvalidUtf8</code>     | 2 | 路径字符串编码无效 |
| <code>ConfigError</code>     | 3 | 配置文件解析错误  |
| <code>GenerationError</code> | 4 | 镜像生成错误    |
| <code>BufferTooSmall</code>  | 5 | 输出缓冲区太小   |

### 4.3 API 函数

#### 4.3.1 `gen_image_from_config`

从配置文件生成镜像数据。

```
1| enum EhsmErrorCode gen_image_from_config(  
2|     const char *config_path,  
3|     uint8_t *output_buf,  
4|     uint32_t *output_len,  
5|     char *err_msg_buf,  
6|     uint32_t err_msg_len  
7| );
```

参数：

- `config_path`: 配置文件路径（C 字符串，必须以 null 结尾）
- `output_buf`: 输出缓冲区（由调用者分配）
- `output_len`: 输入时为缓冲区大小，输出时为实际写入字节数
- `err_msg_buf`: 错误消息缓冲区（可为 NULL）
- `err_msg_len`: 错误消息缓冲区大小

返回值: `EhsmErrorCode` 枚举值。

### 4.3.2 get\_image\_size\_from\_config

预先获取镜像大小, 用于分配缓冲区。

```
1| enum EhsmErrorCode get_image_size_from_config(  
2|     const char *config_path,  
3|     uint32_t *image_size,  
4|     char *err_msg_buf,  
5|     uint32_t err_msg_len  
6| );
```

参数:

- `config_path`: 配置文件路径 (C 字符串)
- `image_size`: 输出镜像大小
- `err_msg_buf`: 错误消息缓冲区 (可为 NULL)
- `err_msg_len`: 错误消息缓冲区大小

返回值: `EhsmErrorCode` 枚举值。

### 4.3.3 get\_imgtool\_lib\_version

获取库版本信息。

```
1| const char *get_imgtool_lib_version(void);
```

返回值: 版本字符串指针 (静态生命周期, 无需释放)。

### 4.3.4 gen\_image\_c\_array\_from\_config

生成镜像并导出为 C 数组源文件。

```
1| enum EhsmErrorCode gen_image_c_array_from_config(  
2|     const char *config_path,  
3|     const char *c_output_path,  
4|     const char *var_name,  
5|     char *err_msg_buf,  
6|     uint32_t err_msg_len  
7| );
```

参数:

- `config_path`: 配置文件路径 (C 字符串, 必须以 null 结尾)
- `c_output_path`: 输出 .c 文件路径 (C 字符串, 必须以 null 结尾)
- `var_name`: 可选的 C 数组变量名 (可为 NULL, 自动从文件名推导)

- `err_msg_buf`: 错误消息缓冲区（可为 NULL）
- `err_msg_len`: 错误消息缓冲区大小

返回值: `EhsmErrorCode` 枚举值。

## 4.4 使用示例

```
1| #include <stdio.h>
2| #include <stdlib.h>
3| #include "imgtool_clib.h"
4|
5| int main(void) {
6|     const char *config_path = "boot_config.toml";
7|     char err_msg[256];
8|     uint32_t image_size = 0;
9|
10|    // 打印库版本
11|    printf("Library version: %s\n", get_imgtool_lib_version());
12|
13|    // 获取镜像大小
14|    if (get_image_size_from_config(
15|        config_path, &image_size, err_msg, sizeof(err_msg)
16|    ) != Success) {
17|        printf("Error: %s\n", err_msg);
18|        return 1;
19|    }
20|
21|    // 分配缓冲区
22|    uint8_t *buffer = malloc(image_size);
23|    uint32_t buf_len = image_size;
24|
25|    // 生成镜像
26|    if (gen_image_from_config(
27|        config_path, buffer, &buf_len, err_msg, sizeof(err_msg)
28|    ) != Success) {
29|        printf("Error: %s\n", err_msg);
30|        free(buffer);
31|        return 1;
32|    }
33|
34|    // 写入文件（需要自行实现文件写入）
35|    FILE *fp = fopen("output.bin", "wb");
36|    if (fp) {
37|        fwrite(buffer, 1, buf_len, fp);
38|        fclose(fp);
39|    }
40|
41|    printf("Image generated: %u bytes\n", buf_len);
```

```
42|     free(buffer);
43|     return 0;
44| }
```

导出 C 数组源文件:

```
1| #include <stdio.h>
2| #include "imgtool_clib.h"
3|
4| int main(void) {
5|     const char *config_path = "boot_config.toml";
6|     char err_msg[256];
7|
8|     // 生成镜像并导出为 C 数组源文件
9|     if (gen_image_c_array_from_config(
10|         config_path,
11|         "boot_image.c",    // 输出 .c 文件路径
12|         NULL,              // 变量名 (NULL 则自动推导)
13|         err_msg, sizeof(err_msg)
14|     ) != Success) {
15|         printf("Error: %s\n", err_msg);
16|         return 1;
17|     }
18|
19|     printf("C array exported successfully\n");
20|     return 0;
21| }
```

## 4.5 编译链接

```
1| gcc -o my_tool my_tool.c -L./lib -limgtool_clib -lpthread -ldl -lm
```

## 5 配置文件格式

配置文件采用 TOML 格式，包含通用配置和模式特定配置。

### 5.1 通用配置 [general]

| 字段                           | 类型     | 必需 | 说明                    |
|------------------------------|--------|----|-----------------------|
| <code>mode</code>            | 字符串    | 是  | "boot" 或 "upgrade"    |
| <code>output</code>          | 字符串    | 是  | 输出文件路径（可被 CLI 参数覆盖）   |
| <code>input_bin</code>       | 字符串    | 是  | 输入固件文件路径（可被 CLI 参数覆盖） |
| <code>image_type</code>      | 整数/字符串 | 是  | 镜像类型（0-3 或名称）         |
| <code>version_counter</code> | 字符串    | 是  | 版本计数器（32 位十六进制）       |
| <code>export_c_array</code>  | 布尔     | 否  | 是否导出 C 数组（默认 false）   |

### 5.2 Boot 模式配置 [boot]

| 字段                      | 类型  | 必需 | 说明                       |
|-------------------------|-----|----|--------------------------|
| <code>sign_key</code>   | 字符串 | 否  | 签名密钥文件路径                 |
| <code>enc_key</code>    | 字符串 | 否  | 加密密钥文件路径                 |
| <code>iv</code>         | 字符串 | 否  | 初始化向量（32 位十六进制），未指定时自动生成 |
| <code>extra_data</code> | 数组  | 否  | 额外数据注入（见下方说明）            |

额外数据注入：

可以在镜像头部的指定偏移量处注入自定义数据：

```
1| [[boot.extra_data]]
2| offset = 600
3| value = "AABCCDD"
```

镜像模式说明：

镜像的生成模式由密钥配置自动确定：

| 配置                                              | 模式     | 说明                     |
|-------------------------------------------------|--------|------------------------|
| 无 <code>sign_key</code>                         | 裸镜像    | 仅包含有效标志、裸镜像标志和代码长度     |
| 有 <code>sign_key</code> ，无 <code>enc_key</code> | 明文签名镜像 | 签名但不加密（plain_flag = 1） |
| 有 <code>sign_key</code> 和 <code>enc_key</code>  | 加密签名镜像 | 签名并加密                  |
| 有 <code>enc_key</code> ，无 <code>sign_key</code> | 错误     | 加密必须配合签名使用             |

镜像类型说明：

镜像类型可以使用数字或字符串名称：

| 数字 | 字符串名称           | 说明                |
|----|-----------------|-------------------|
| 0  | "ehsm-ehsmkey"  | eHSM 固件使用 eHSM 密钥 |
| 1  | "soc-socket"    | SoC 固件使用 SoC 密钥   |
| 2  | "soc-ehsmkey"   | SoC 固件使用 eHSM 密钥  |
| 3  | "patch-ehsmkey" | eHSM Patch 镜像     |

### Boot 配置示例:

仅签名（不加密）:

```

1| [general]
2| mode = "boot"
3| output = "boot_signed.bin"
4| input_bin = "firmware.bin"
5| image_type = 0
6| version_counter = "FFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFF"
7|
8| [boot]
9| sign_key = "keys/aes128.toml"

```

签名并加密（使用字符串类型名称）:

```

1| [general]
2| mode = "boot"
3| output = "boot_encrypted.bin"
4| input_bin = "firmware.bin"
5| image_type = "ehsm-ehsmkey"
6| version_counter = "FFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFF"
7|
8| [boot]
9| sign_key = "keys/sm2.toml"
10| enc_key = "keys/sm4.toml"
11| iv = "5ED24925293ACB755C50D8B263A2A3CC"

```

裸镜像（无签名密钥）:

```

1| [general]
2| mode = "boot"
3| output = "naked.bin"
4| input_bin = "firmware.bin"
5| image_type = 0
6| version_counter = "FFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFF"
7|
8| [boot]
9| # No sign_key = naked image

```

带额外数据注入：

```
1| [general]
2| mode = "boot"
3| output = "boot_with_extra.bin"
4| input_bin = "firmware.bin"
5| image_type = 0
6| version_counter = "FFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFF"
7|
8| [boot]
9| sign_key = "keys/aes128.toml"
10| enc_key = "keys/aes128.toml"
11|
12| [[boot.extra_data]]
13| offset = 600
14| value = "AABCCDD"
```

## 5.3 Upgrade 模式配置 [upgrade]

Upgrade 镜像采用双层结构：

- Boot 层：内层，仅签名（固件数据不加密）
- Upgrade 层：外层，签名并加密（加密整个 Boot 层）

**重要说明：**在 Upgrade 镜像中，Boot 层的固件数据始终以明文形式存储，仅进行签名处理。外层的 Upgrade 层会对整个 Boot 层（包含头部和固件数据）进行加密。

### 5.3.1 Boot 层配置 [upgrade.boot\_layer]

| 字段                      | 类型  | 必需 | 说明                            |
|-------------------------|-----|----|-------------------------------|
| <code>sign_key</code>   | 字符串 | 是  | Boot 层签名密钥文件路径                |
| <code>plain</code>      | 布尔  | 否  | Boot 镜像输出模式标志（默认 false，见下方说明） |
| <code>extra_data</code> | 数组  | 否  | 额外数据注入                        |

**plain 配置说明：**

`plain` 字段用于控制 eHSM 设备在升级过程中的行为，而非本工具的生成行为：

- 当设置为 `true` 时，表示 eHSM 在完成升级后，将 Boot 镜像以明文形式写入存储
- 当设置为 `false`（默认）时，表示 eHSM 在完成升级后，会对 Boot 镜像进行加密后再写入存储

此配置值会写入 Boot 层的头部信息中，供 eHSM 设备读取和处理。本工具仅将此标志写入镜像头部，不会根据此配置值执行任何加密操作。

### 5.3.2 Upgrade 层配置 [upgrade.upgrade\_layer]

| 字段                    | 类型  | 必需 | 说明                |
|-----------------------|-----|----|-------------------|
| <code>sign_key</code> | 字符串 | 是  | Upgrade 层签名密钥文件路径 |



| 字段                      | 类型  | 必需 | 说明                       |
|-------------------------|-----|----|--------------------------|
| <code>enc_key</code>    | 字符串 | 是  | Upgrade 层加密密钥文件路径        |
| <code>iv</code>         | 字符串 | 否  | 初始化向量（32 位十六进制），未指定时自动生成 |
| <code>extra_data</code> | 数组  | 否  | 额外数据注入                   |

### Upgrade 配置示例：

```
1| [general]
2| mode = "upgrade"
3| output = "upgrade_image.bin"
4| input_bin = "firmware.bin"
5| image_type = 0
6| version_counter = "FFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFF"
7|
8| [upgrade.boot_layer]
9| sign_key = "keys/aes128.toml"
10| plain = true
11|
12| [upgrade.upgrade_layer]
13| sign_key = "keys/rsa2048.toml"
14| enc_key = "keys/aes128.toml"
15| # iv = "5ED24925293ACB755C50D8B263A2A3CC" # 可选，未指定时自动生成
```

### 带额外数据注入的示例：

```
1| [general]
2| mode = "upgrade"
3| output = "upgrade_image.bin"
4| input_bin = "firmware.bin"
5| image_type = 0
6| version_counter = "FFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFF"
7|
8| [upgrade.boot_layer]
9| sign_key = "keys/aes128.toml"
10|
11| [[upgrade.boot_layer.extra_data]]
12| offset = 600
13| value = "AABCCDD"
14|
15| [upgrade.upgrade_layer]
16| sign_key = "keys/rsa2048.toml"
17| enc_key = "keys/aes128.toml"
18|
19| [[upgrade.upgrade_layer.extra_data]]
20| offset = 700
21| value = "11223344"
```

## 6 密钥文件格式

密钥文件采用 TOML 格式，根据算法类型有不同的结构。

### 6.1 对称密钥

对称密钥（AES、SM4）格式简单，包含类型和密钥值。

**AES128 密钥**（16 字节 = 32 位十六进制）：

```
1| type = "aes128"
2| value = "32BB3737FB468C4B6AC9D84C1661737D"
```

**AES256 密钥**（32 字节 = 64 位十六进制）：

```
1| type = "aes256"
2| value = "32BB3737FB468C4B6AC9D84C1661737D32BB3737FB468C4B6AC9D84C1661737D"
```

**SM4 密钥**（16 字节 = 32 位十六进制）：

```
1| type = "sm4"
2| value = "32BB3737FB468C4B6AC9D84C1661737D"
```

### 6.2 非对称密钥

非对称密钥包含公钥和私钥组件，使用 `[value]` 节定义。

#### 6.2.1 RSA 密钥

RSA 密钥包含三个组件：

- `e`：公钥指数，必须是 64 字节
- `n`：模数
- `d`：私钥指数

**RSA2048 密钥**：

```
1| type = "rsa2048"
2|
3| [value]
4| e = "00000000...010001"
5| n = "A722EB63...28503"
6| d = "09BAAFA7...6E41"
```

**RSA3072 密钥**：

```
1| type = "rsa3072"
2|
3| [value]
4| e = "00000000...010001"
5| n = "..."
6| d = "..."
```

### 6.2.2 ECC 密钥

ECC256 (secp256r1) 密钥包含:

- `public`: 公钥点坐标 (X || Y, 64 字节)
- `private`: 私钥标量 (32 字节)

```
1| type = "ecc256"
2|
3| [value]
4| public = "370885247EAB66499FD554ADF24B8481...535878"
5| private = "415767176EBE1F38B399EC637CED52E8F501DBAD2A981F3BD9222A3CDE8DFEA2"
```

### 6.2.3 SM2 密钥

SM2 密钥格式与 ECC256 类似:

- `public`: 公钥点坐标 (带 04 前缀, 65 字节)
- `private`: 私钥标量 (32 字节)

```
1| type = "sm2"
2|
3| [value]
4| public = "0428F01618BAC2FA8D69FC5FC4DD18207476...12F9"
5| private = "06DB4C16F922728B991C10442E8638E478E2266EC96C1F13A7FCE0F8A80D911D"
```

## 7 发布包目录结构

发布包为 zip 压缩文件，解压后的目录结构如下：

```
1| imgtool-<版本号>/
2| |—— x86_64-windows/
3| |   |—— bin/
4| |     |—— imgtool.exe      # Windows CLI 工具
5| |   |—— include/
6| |     |—— imgtool_clib.h   # C 库头文件
7| |     |—— lib/
8| |       |—— libimgtool_clib.a # C 静态库
9| |—— x86_64-linux/
10| |   |—— bin/
11| |     |—— imgtool          # Linux CLI 工具
12| |   |—— include/
13| |     |—— imgtool_clib.h   # C 库头文件
14| |     |—— lib/
15| |       |—— libimgtool_clib.a # C 静态库
16| |—— configs-demo/         # 配置文件示例
17| |   |—— boot/              # Boot 模式配置示例
18| |   |—— upgrade/           # Upgrade 模式配置示例
19| |     |—— keys/            # 密钥文件示例
20| |—— c-src-demo/            # C 库使用示例代码
21| |—— user_guide.pdf         # 用户手册
22| |—— changelog.md           # 变更日志
```

### 7.1 目录说明

| 目录/文件           | 说明             |
|-----------------|----------------|
| x86_64-windows/ | Windows 平台工具和库 |
| x86_64-linux/   | Linux 平台工具和库   |
| configs-demo/   | 配置文件和密钥文件示例    |
| c-src-demo/     | C 库集成示例代码      |
| user_guide.pdf  | 本用户手册          |
| changelog.md    | 版本变更记录         |

## A 镜像结构

### A.1 Boot 镜像结构

Boot 镜像由 1024 字节头部和固件数据组成：

| 1  | 偏移量   | 大小  | 内容                |
|----|-------|-----|-------------------|
| 2  | 0x000 | 256 | 签名数据              |
| 3  | 0x100 | 320 | 公钥区域              |
| 4  | 0x240 | 16  | 初始化向量 (IV)        |
| 5  | 0x250 | 4   | 镜像类型              |
| 6  | 0x254 | 16  | 版本计数器             |
| 7  | 0x264 | 4   | 固件长度              |
| 8  | 0x268 | 4   | 有效标志 (0x8E97645D) |
| 9  | ...   |     |                   |
| 10 | 0x400 | N   | 固件数据 (可加密)        |

### A.2 Upgrade 镜像结构

Upgrade 镜像包含外层头部和加密的 Boot 镜像：

| 1  | 偏移量   | 大小  | 内容                |
|----|-------|-----|-------------------|
| 2  | 0x000 | 256 | Upgrade 层签名       |
| 3  | 0x100 | 320 | 公钥区域              |
| 4  | 0x240 | 16  | 初始化向量 (IV)        |
| 5  | 0x250 | 4   | 镜像类型              |
| 6  | 0x254 | 16  | 版本计数器             |
| 7  | 0x264 | 4   | 内层镜像长度            |
| 8  | 0x268 | 4   | 有效标志 (0x71689BA2) |
| 9  | ...   |     |                   |
| 10 | 0x400 | M   | 加密的 Boot 镜像       |

## B 版本计数器

版本计数器为 16 字节（128 位）的十六进制字符串，用于防回滚保护，且以小端存储。设备在启动或升级时会验证版本计数器，拒绝加载版本号较低的镜像。

建议使用递增的版本号，每次改变 1 个 bit，例如：

- 对于 OTP 值从 0 变 1 的系统
  - 初始版本：`01000000000000000000000000000000`
  - 第二版本：`03000000000000000000000000000000`
  - 最大版本：`FFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFF`
- 对于 OTP 值从 1 变 0 的系统
  - 初始版本：`FEFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFF`
  - 第二版本：`FCFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFF`
  - 最大版本：`00000000000000000000000000000000`

## C 初始化向量 (IV)

初始化向量为 16 字节（128 位）的十六进制字符串，用于 AES-CBC 或 SM4-CBC 加密。

注意事项：

- 每个镜像应使用唯一的 IV
- IV 不需要保密，但应确保随机性
- 如果配置文件中未指定 IV，工具会自动生成随机 IV
- 手动指定 IV 时，必须为 32 个十六进制字符（16 字节）